Workshop Objectives and Techniques

The primary objective of this workshop was to build a neural network capable of classifying images as either "chihuahua" or "muffin," demonstrating the complete workflow of deep learning-based image classification using PyTorch [1]. The workshop covered defining a feed-forward neural network, preparing image datasets with data loaders, training the model, and evaluating predictions on validation data. Core techniques included data loading with torchvision. datasets.ImageFolder, tensor transformations (resize, normalization, batching), supervised training with cross-entropy loss, and optimization via stochastic gradient descent [1][2].

The workshop exemplified a standard computer vision pipeline: preparing labeled data, designing model architecture, training through mini-batch gradient descent, and evaluating generalization on unseen validation images [2][3]. Although the model used fully connected layers over flattened pixels rather than a full convolutional architecture, it provided an accessible introduction paralleling more advanced CNNs used in production systems.

Key Concepts Learned

Image Classification: The fundamental task of assigning a single class label to each input image, one of the foundational problems in computer vision that underpins object detection, segmentation, and activity recognition [3][4]. The workshop demonstrated how a small, curated dataset (120 training images, 30 validation images) can train a model to distinguish visually similar categories.

Neural Network Architecture: The model consisted of multiple fully connected layers with ReLU activations, ending in a two-dimensional output for binary classification. Training employed cross-entropy loss to measure prediction error and SGD optimizer to update weights via backpropagation [1][2]. The training loop demonstrated typical deep learning patterns: separate train/validation phases, batched iteration, and tracking loss and accuracy per epoch.

Transfer Learning Foundations: While not fully implemented, the workshop introduced the concept of transfer learning, where networks pre-trained on large datasets like ImageNet are reused and fine-tuned for new tasks [2][5]. This approach significantly reduces training time and improves performance, especially with limited labeled data.

Challenges and Solutions

Hyperparameter Selection: Choosing appropriate input dimensions (128×128), batch size (8), and learning rate (0.1) required experimentation. Initial configurations sometimes lead to slow convergence or training instability. Adjusting the learning rate downward or switching to the Adam optimizer helped stabilize training dynamics [2].

Training Dynamics: With a high initial learning rate, the model occasionally overshot optimal solutions or converged slowly. Monitoring both training and validation accuracy

revealed when overfitting occurred, emphasizing the importance of validation metrics over training loss alone [1][5].

Data Pipeline Configuration: Ensuring the directory structure matches Image Folder expectations and applying consistent transforms to both training and validation data was essential. Shape mismatches between tensors initially caused errors, resolved by verifying tensor dimensions and ensuring all tensors reside on the same device (CPU or GPU) [1].

Insights About Machine Learning

The workshop highlighted how sensitive neural networks are to data preparation—normalization ranges, resize strategies, and shuffling significantly impact training behavior and final accuracy [2]. The clear separation between training and validation data proved essential for estimating generalization; models can memorize small training sets while performing poorly on new images [5].

Deep learning models excel at capturing subtle visual differences that are difficult to encode with handcrafted features. Neural networks automatically learn hierarchical features—from edges in early layers to abstract shapes in deeper layers, making them ideal for image classification [4][5]. The iterative experimentation process demonstrated that systematic tweaking of architecture and hyperparameters, combined with careful metric observation, drives progress more effectively than any single configuration [2].

Real-World Applications

Though playful, the chihuahua-muffin problem mirrors serious applications requiring distinction between visually similar classes. In healthcare, image classification models analyze radiology images and histology slides to detect tumors and abnormalities [3][4]. Manufacturing uses automated visual inspection systems for quality control, while autonomous vehicles rely on CNN-based systems to classify pedestrians, vehicles, and traffic signs for safe navigation [3][4].

Security applications employ face recognition and biometric classification built on robust models trained on large datasets. Additional domains include land-use mapping from satellite imagery, retail analytics, content moderation, and recommendation systems [3][4]. Transfer learning extends these applications by adapting general pre-trained models to specialized domain-medical imaging, satellite analysis, industrial inspection—often with relatively small domain-specific datasets [2][5].

Personal Reflections

Working through the notebook provided concrete, hands-on experience with modern deep learning tools rather than abstract theory. Watching the training loop progress over epochs and visualizing prediction confidences on validation images created an intuitive understanding of how neural networks learn from data [1]. The model's confidence scores revealed that predictions are rarely perfectly certain; output probabilities reflect ambiguity in visually confusing cases.

The exercise emphasized the importance of debugging and iteration in machine learning. Rather than treating hyperparameters as fixed, the workshop encouraged modifying layer sizes, optimizer settings, and data transforms to observe performance changes [2]. This experimentation-built intuition about which modifications improve generalization versus encouraging overfitting. The reproducible code structure—clear separation of model, data, training loop, and evaluation—proved vital when exploring different configurations.

Connecting this focused problem to broader applications made the workshop feel relevant beyond the toy example. The core ideas—data loaders, neural architectures, loss functions, optimizers, and held-out evaluation—apply directly to serious image classification tasks in healthcare, manufacturing, and transportation [3][4]. The experience provides a solid foundation for further study of convolutional neural networks and transfer learning in computer vision.

References

[1] GitHub. (2023). mlatsjsu/workshop-chihuahua-vs-muffin. https://github.com/mlatsjsu/workshop-chihuahua-vs-muffin

[2] PyTorch. (2022). Transfer Learning for Computer Vision Tutorial. https://pytorch.org/tutorials/beginner/transfer_learning_tutorial.html

[3] Viso.ai. (2025). Mastering Image Classification with CNN & Edge AI. https://viso.ai/computer-vision/image-classification/

[4] Onix Systems. (2025). 26 Real-World Image Classification Use Cases in 6 Industries. https://onix-systems.com/blog/image-classification-use-cases

[5] Kandel, I., & Castelli, M. (2020). Transfer learning with deep convolutional neural networks for image classification. *Applied Sciences*, 10(18). https://doi.org/10.3390/app10186820